FUNDAMENTOS DA WEB MODERNA

APIs, Arquitetura REST e Retornos JSON

Como sistemas distribuídos comunicam dados através do protocolo HTTP de maneira rápida, padronizada e segura.

CONCEITO ESSENCIAL

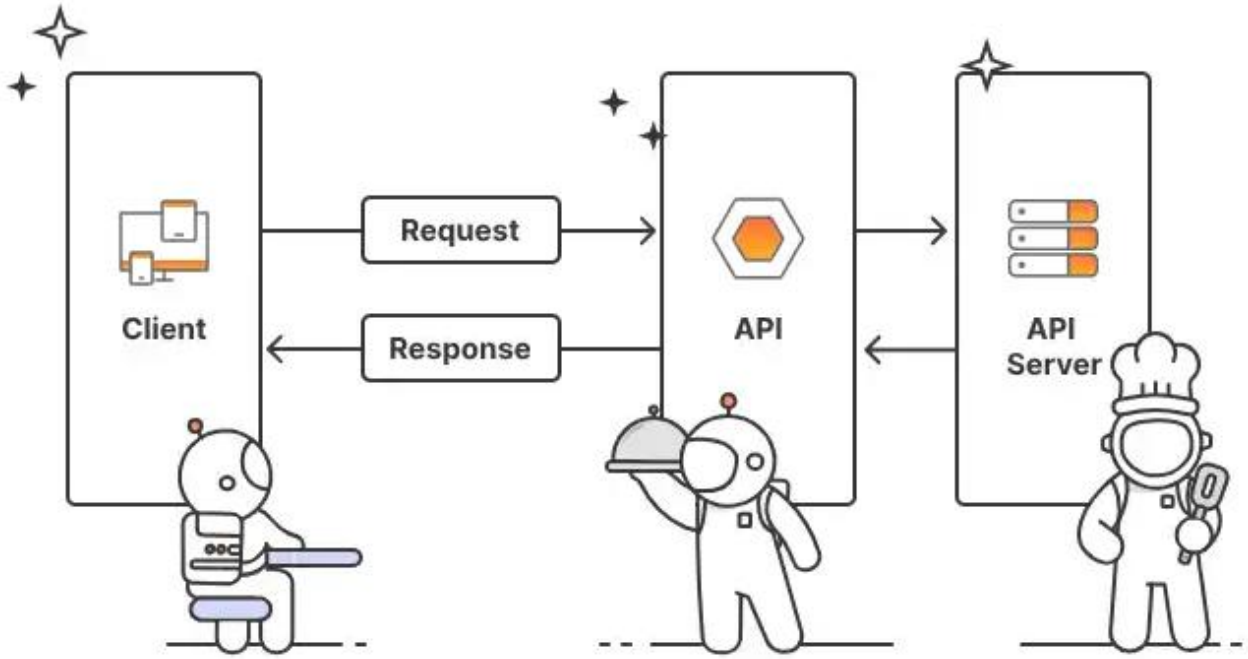
O que é uma API?

Interface de Programação de Aplicações.

Um contrato de comunicação que permite a dois softwares trocarem dados e serviços sem conhecerem o código interno um do outro.

> Analogia: o garçom entre o cliente e a cozinha.





ARQUITETURA REST

Pilares de uma REST API

Cliente-Servidor
Interface e banco desacoplados.

Stateless
Cada requisição é autossuficiente.

Recursos Padronizados
Identificação uniforme via URIs.

VERBOS HTTP

Ações e Verbos Padronizados

GET

Leitura de Dados

Consulta registros sem alterar estado.

POST

Criação de Registro

Envia dados para criar novo recurso.

PUT / PATCH

Atualização

Modifica recursos existentes no servidor.

DELETE

Exclusão

Remove recursos do banco de dados.

RESPOSTAS DO SERVIDOR

Status Codes Essenciais

2xx

Sucesso

200 OK: Operação concluída.
201 Created: Criado com êxito.

4xx

Erro do Cliente

400 Bad Request: Sintaxe inválida.
404 Not Found: Não localizado.

5xx

Erro do Servidor

500 Internal Error: Falha no código.
503 Unavailable: Serviço fora do ar.

FORMATO DE DADOS

O que é JSON?

{ JSON }

JavaScript Object Notation

Padrão universal de troca de texto estruturado baseado em pares de chave e valor. É leve e legível tanto por humanos quanto por máquinas.

ESTRUTURA DE DADOS

Anatomia de um Payload JSON

```json
{
  "id": 550,
  "titulo": "Clube da Luta",
  "emCartaz": true,
  "nota": 8.4,
  "generos": [ "Drama", "Suspense" ],
  "distribuidor": {
    "nome": "Fox 2000 Pictures",
    "pais": "BR"
  }
}
```

TIPAGEM PRIMITIVA

Tipos Aceitos pelo Formato

Textos e Números

String: Entre aspas duplas.

Number: Inteiros e decimais sem aspas.

Valores Lógicos e Nulos

Boolean: true ou false literais.

Null: Ausência declarada de valor.

Coleções e Objetos

Array [ ]: Listas ordenadas de itens.

Object { }: Agrupamentos de propriedades.

INTEGRAÇÃO NO ASP.NET MVC

Desserializando JSON com System.Text.Json

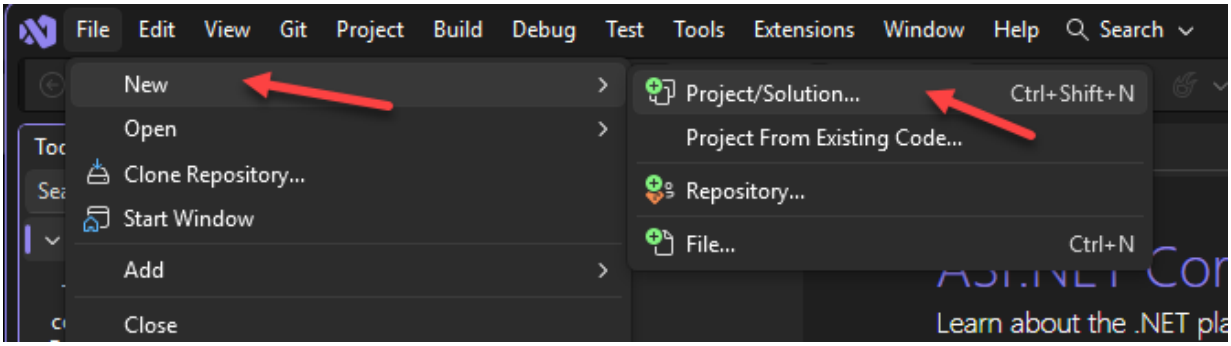
```
// 1. Faz a requisição HTTP GET para a REST API
var resposta = await clienteHttp.GetAsync("https://api.exemplo.com/filmes");
var jsonTexto = await resposta.Content.ReadAsStringAsync();

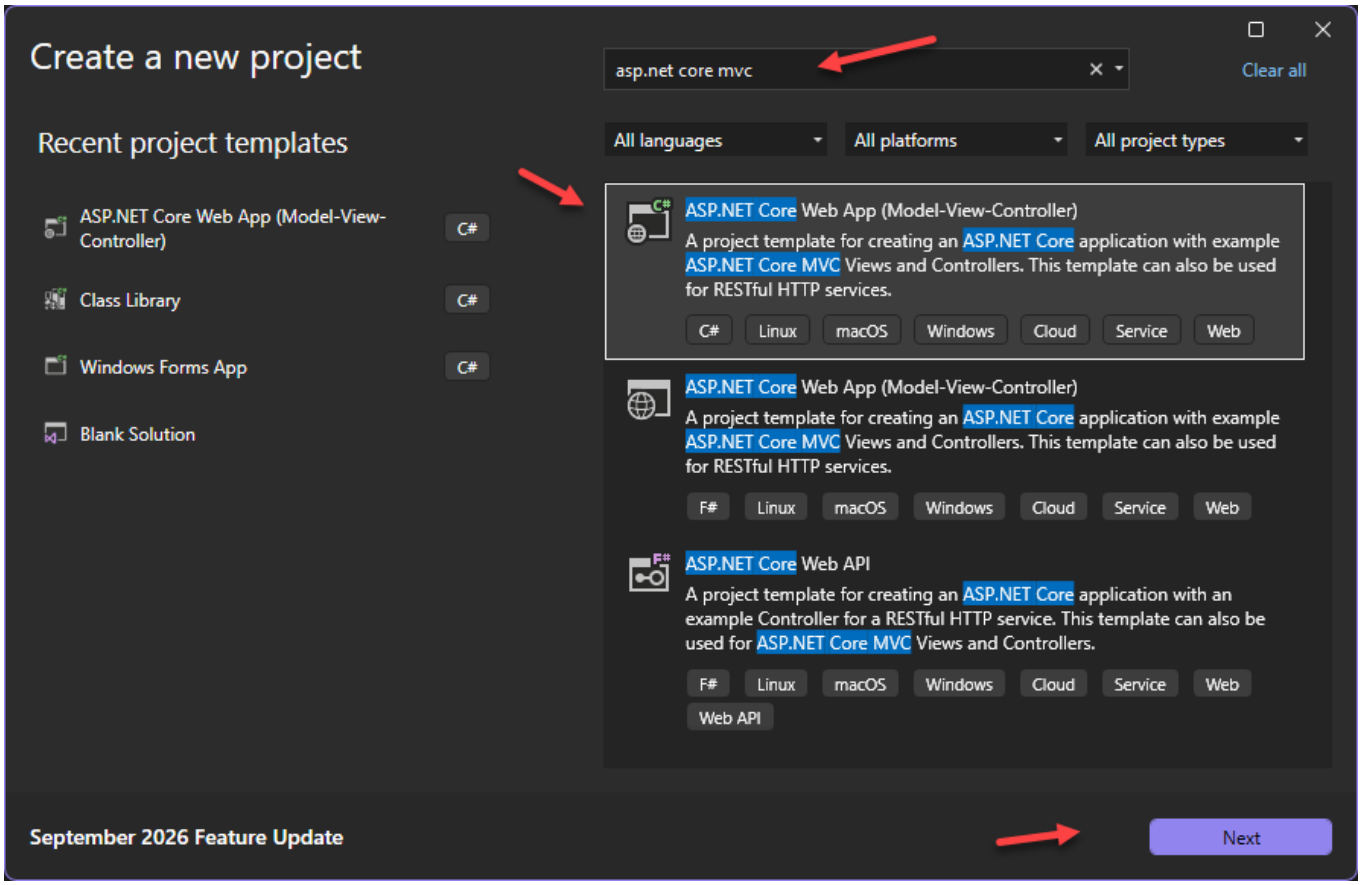
// 2. Converte o texto JSON em objetos fortemente tipados em C#
var dados = JsonSerializer.Deserialize<List<FilmeModel>>(jsonTexto, new JsonSerializerOptions
{
    PropertyNameCaseInsensitive = true
});
```

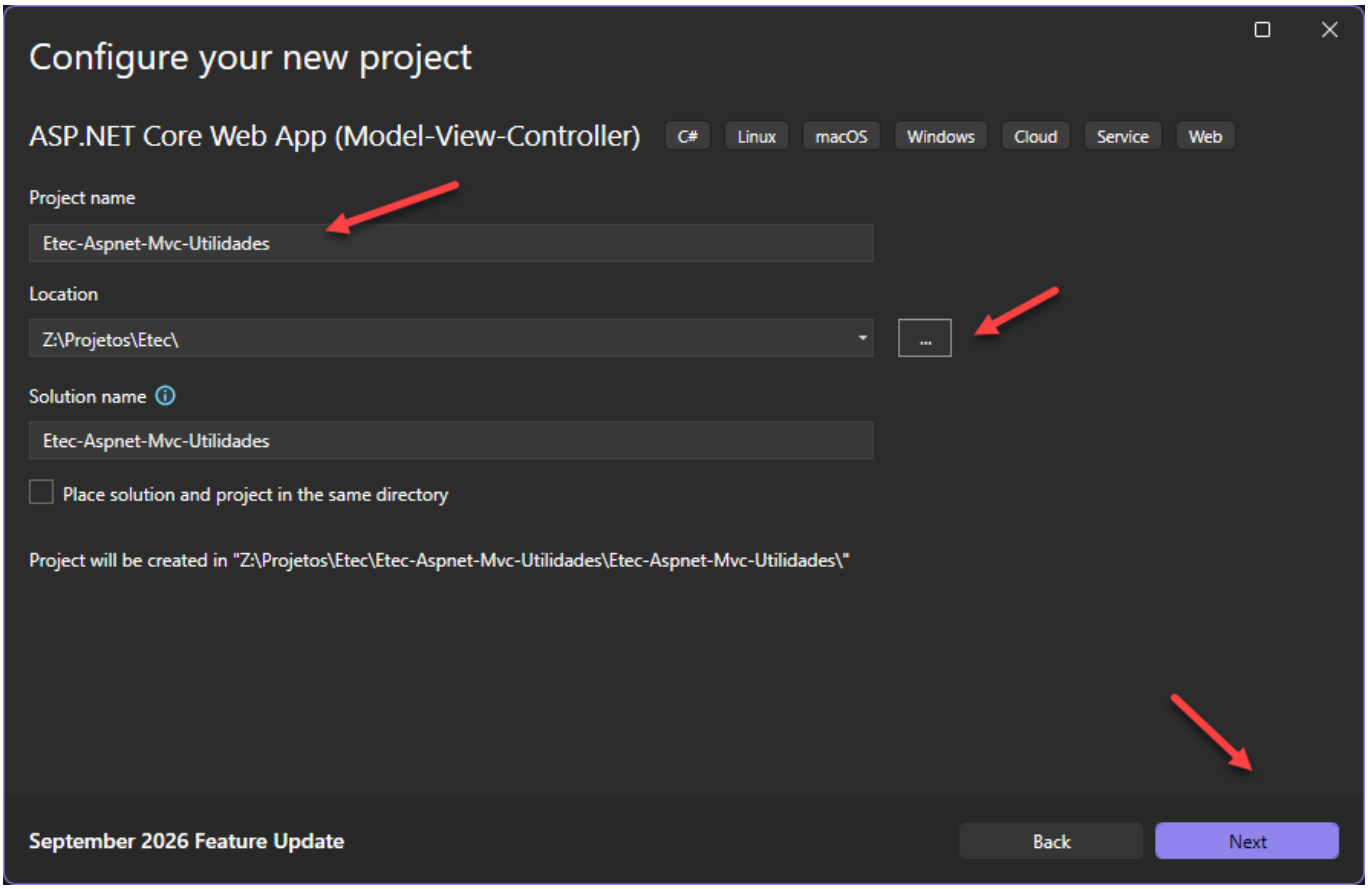
Atividade: Acessando Apis

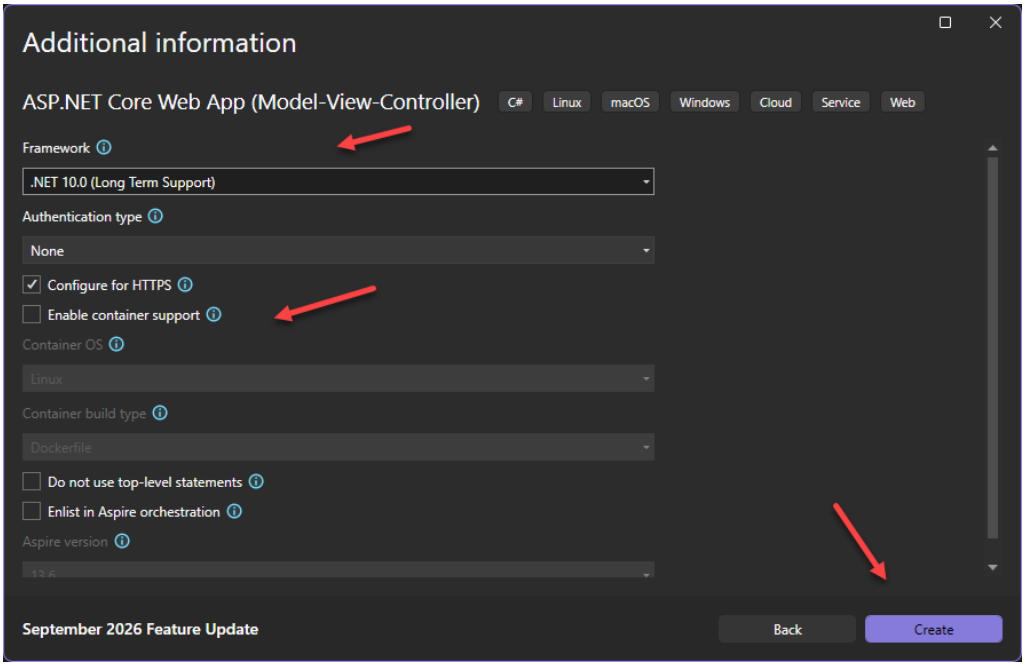
Acesse o Visual Studio e crie um novo Projeto ASP.Net Core MVC



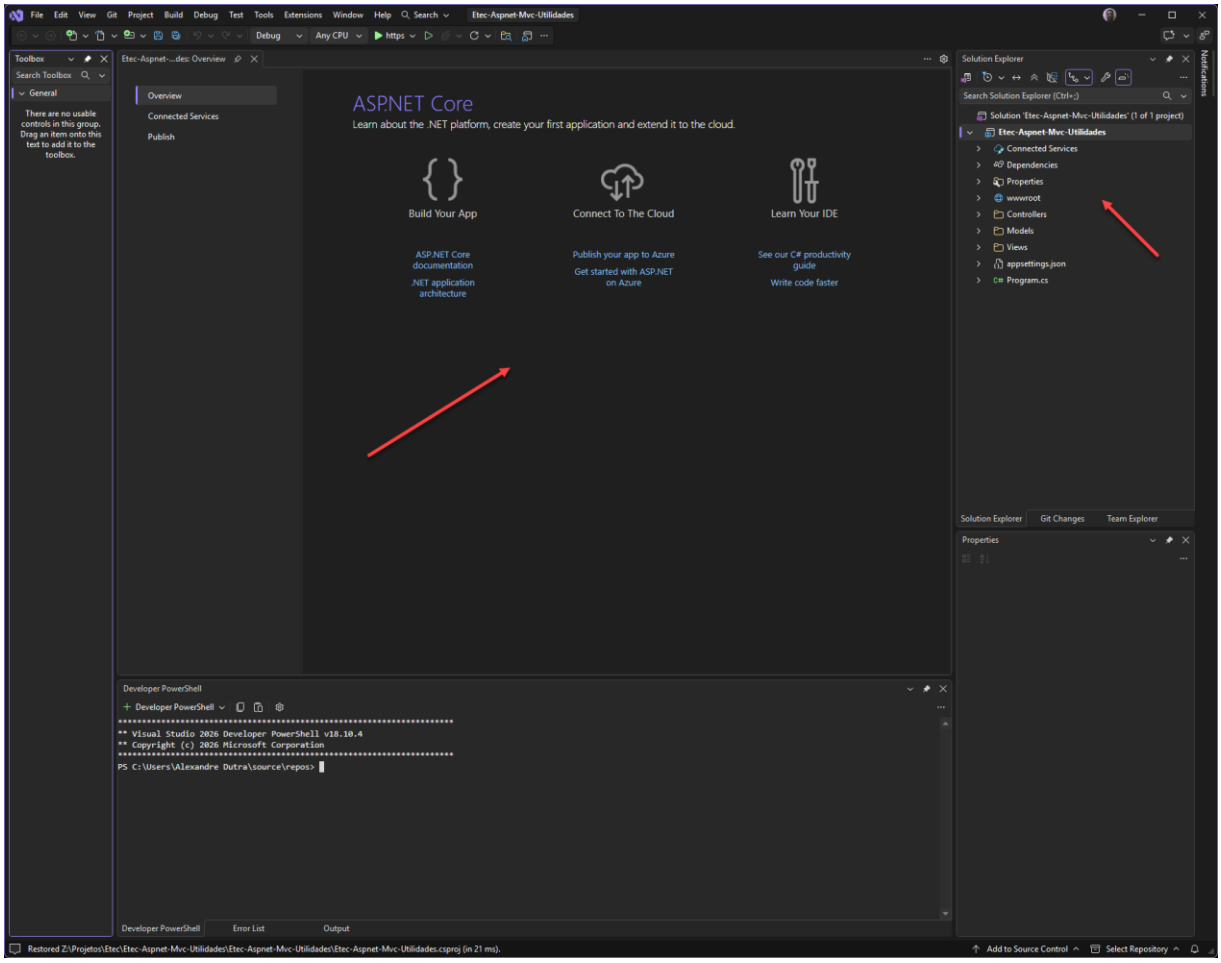



Create a new project
asp.net core mvc
Clear all
Recent project templates
All languages
All platforms
All project types
ASP.NET Core Web App (Model-View-Controller)
C#
Class Library
C#
Windows Forms App
C#
Blank Solution
ASP.NET Core Web App (Model-View-Controller)
A project template for creating an ASP.NET Core application with example ASP.NET Core MVC Views and Controllers. This template can also be used for RESTful HTTP services.
C# Linux macOS Windows Cloud Service Web
ASP.NET Core Web App (Model-View-Controller)
A project template for creating an ASP.NET Core application with example ASP.NET Core MVC Views and Controllers. This template can also be used for RESTful HTTP services.
F# Linux macOS Windows Cloud Service Web
ASP.NET Core Web API
A project template for creating an ASP.NET Core application with an example Controller for a RESTful HTTP service. This template can also be used for ASP.NET Core MVC Views and Controllers.
F# Linux macOS Windows Cloud Service Web
Web API
September 2026 Feature Update
Next

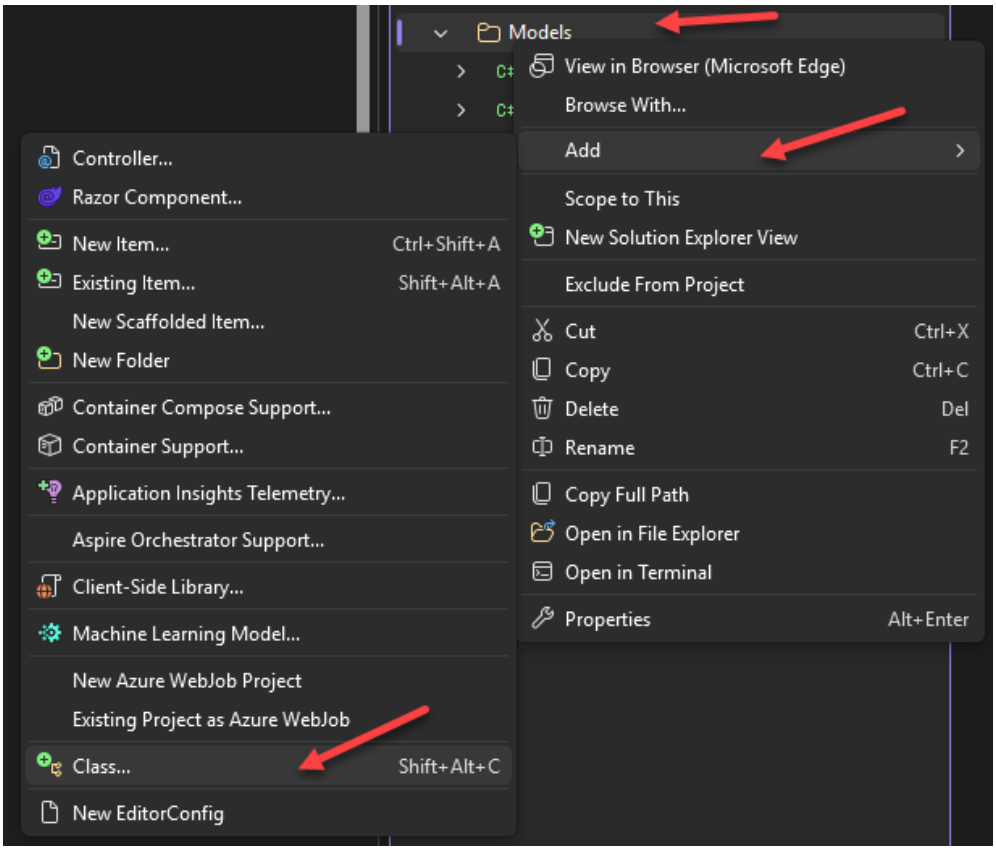

Configure your new project
ASP.NET Core Web App (Model-View-Controller)
C# Linux macOS Windows Cloud Service Web
Project name
Etec-Aspnet-Mvc-Utilidades
Location
Z:\Projetos\Etec\
...
Solution name
Etec-Aspnet-Mvc-Utilidades
Place solution and project in the same directory
Project will be created in "Z:\Projetos\Etec\Etec-Aspnet-Mvc-Utilidades\Etec-Aspnet-Mvc-Utilidades\"
September 2026 Feature Update
Back
Next

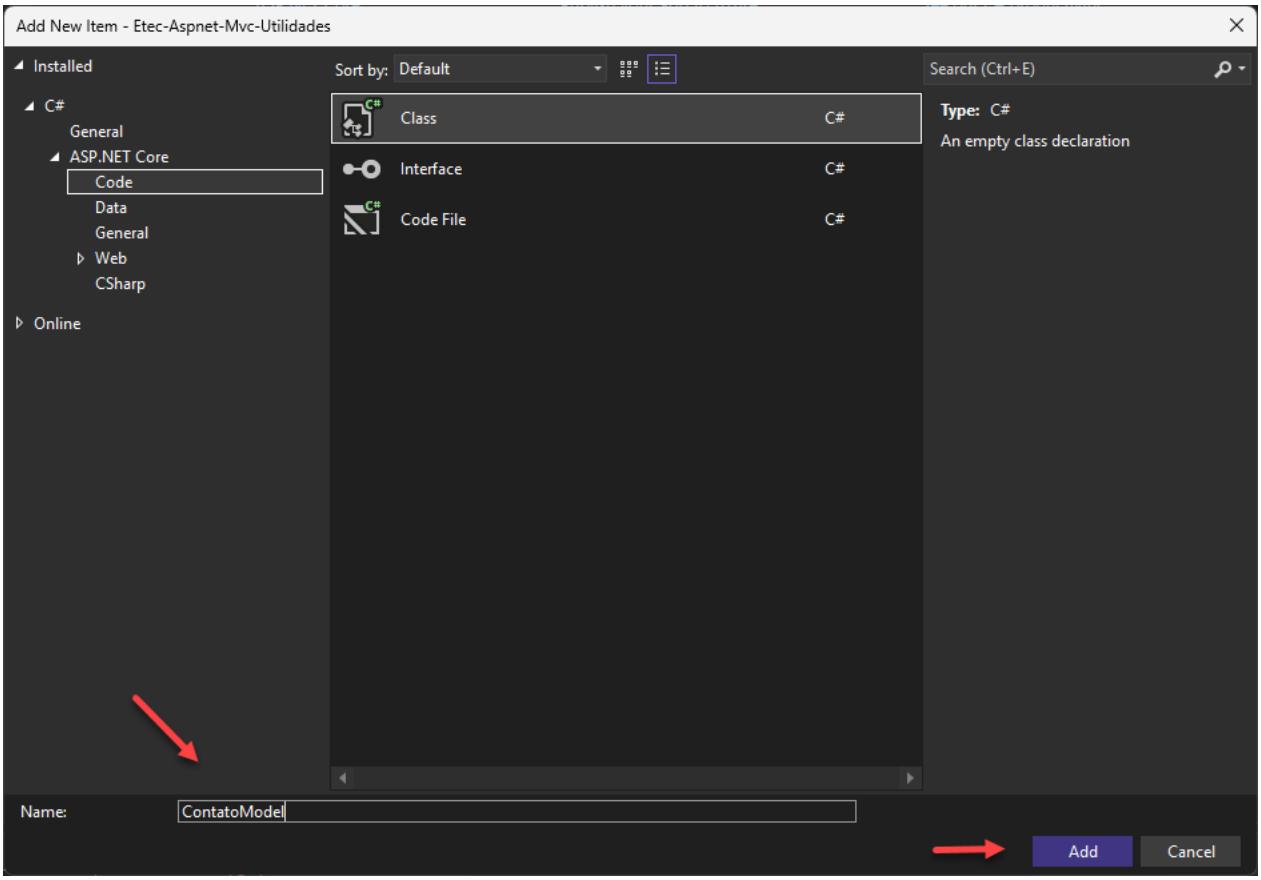


Um novo projeto será criado e exibido no ambiente do Visual Studio com a estrutura padrão que já conhecemos.

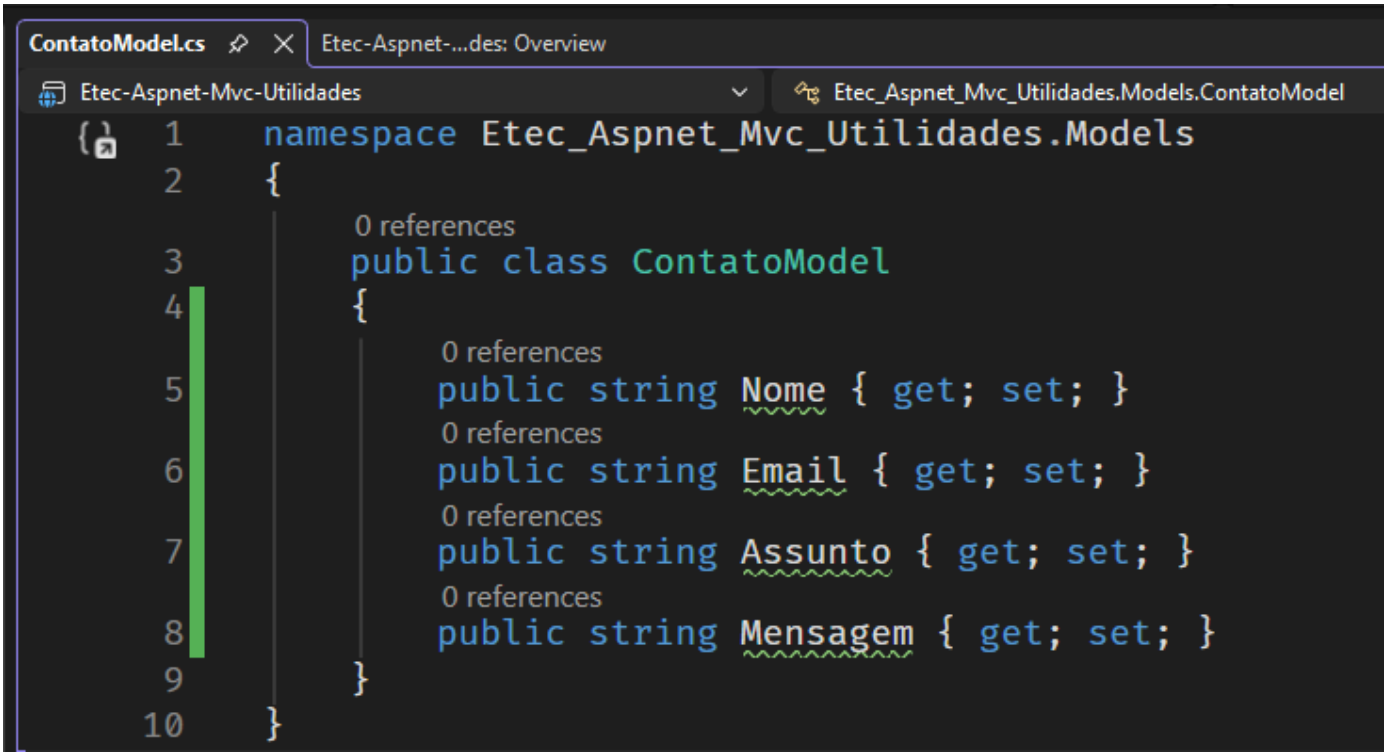


Agora vamos adicionar uma nova classe chamada ContatoModel, na Pasta Models, para isso clique o botão direito na pasta Models e acesse as opções

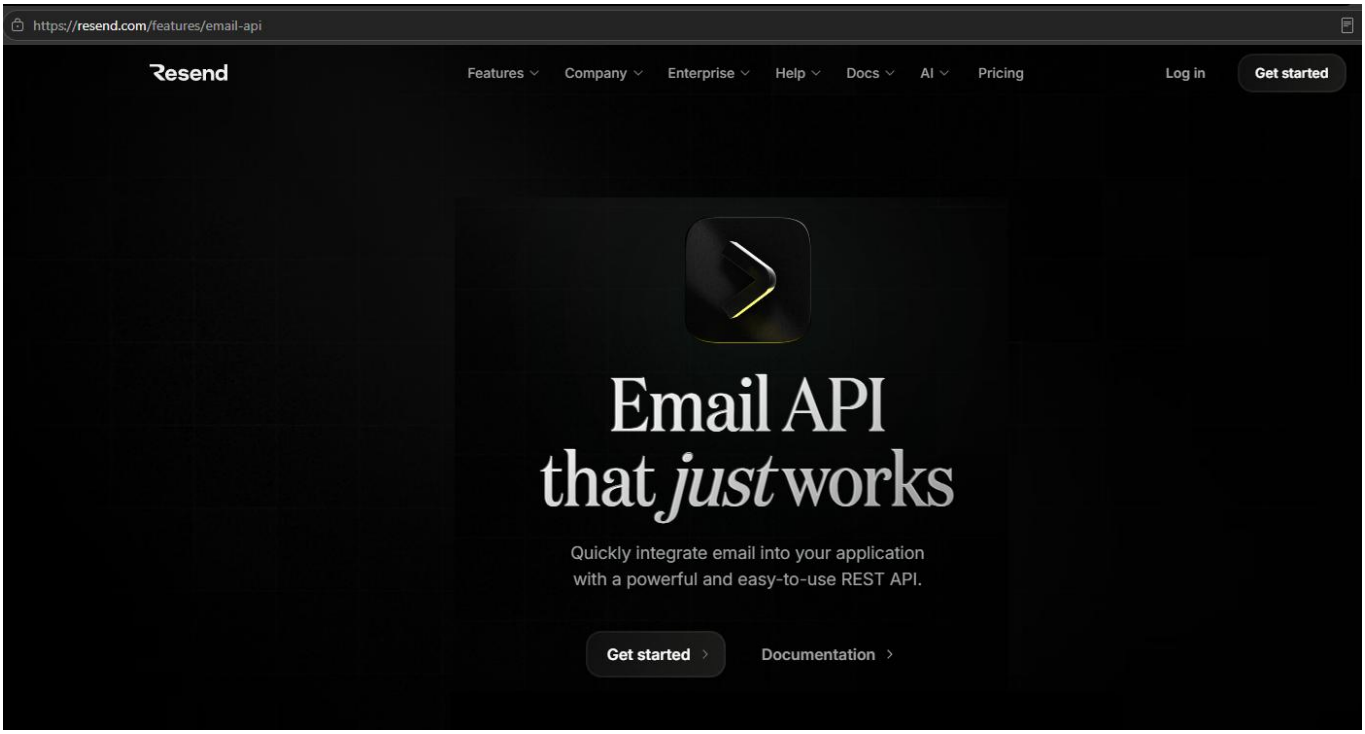




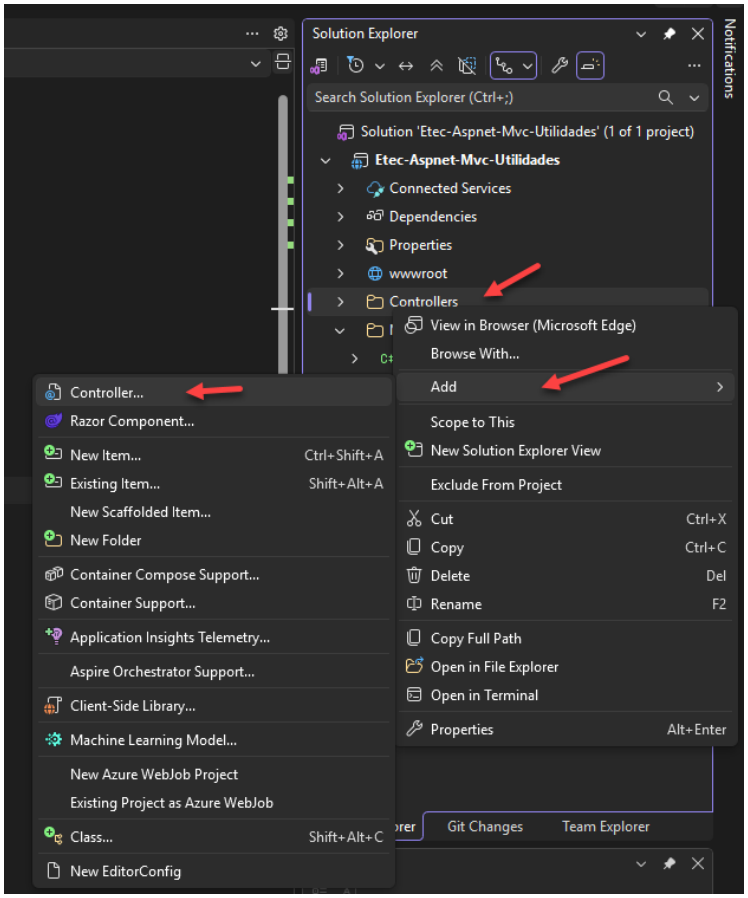
A classe deve ficar da seguinte forma

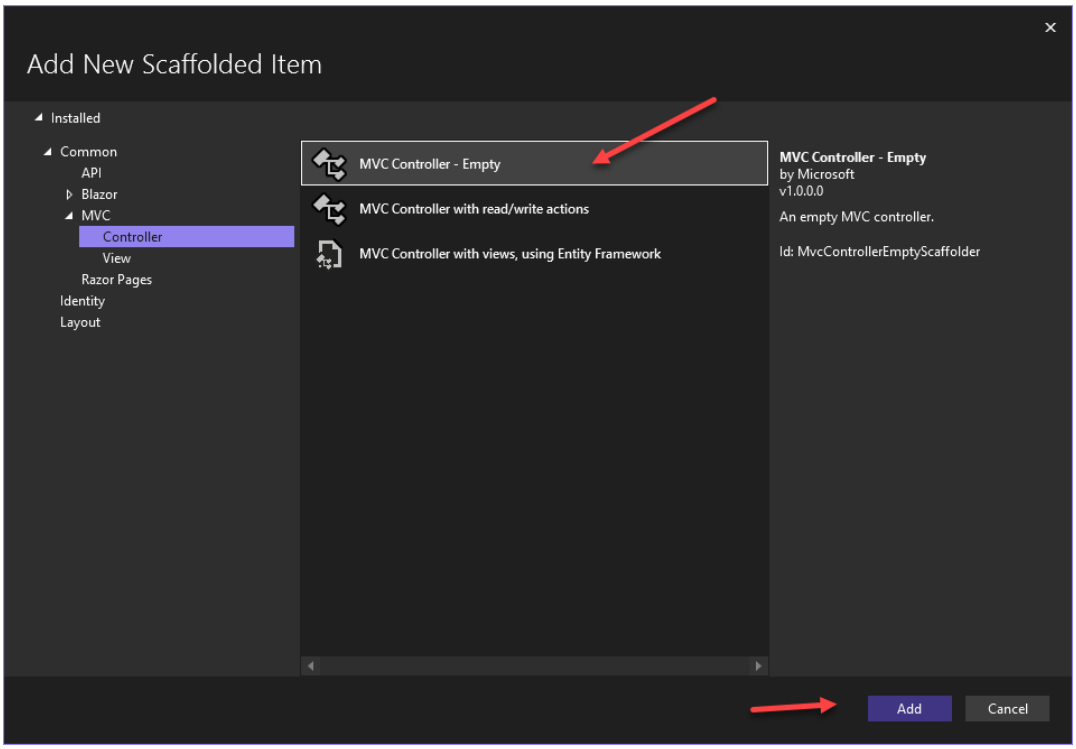


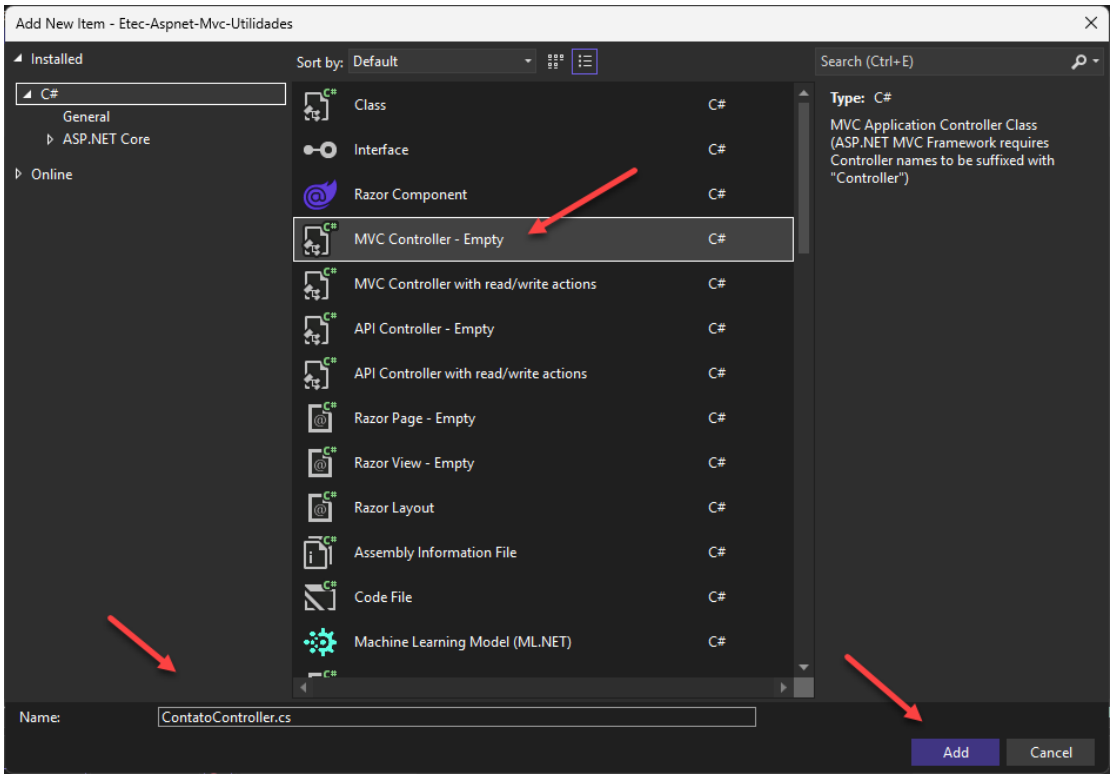
Para o exemplo iremos utilizar um serviço de **API** de emails chamado **RESEND,** para acessar vamos utilizar uma chave que foi criada para testes: ApiKey=re_H6hX2HqX_9B4X9v7iqTXkmcgeNyVm7kfU



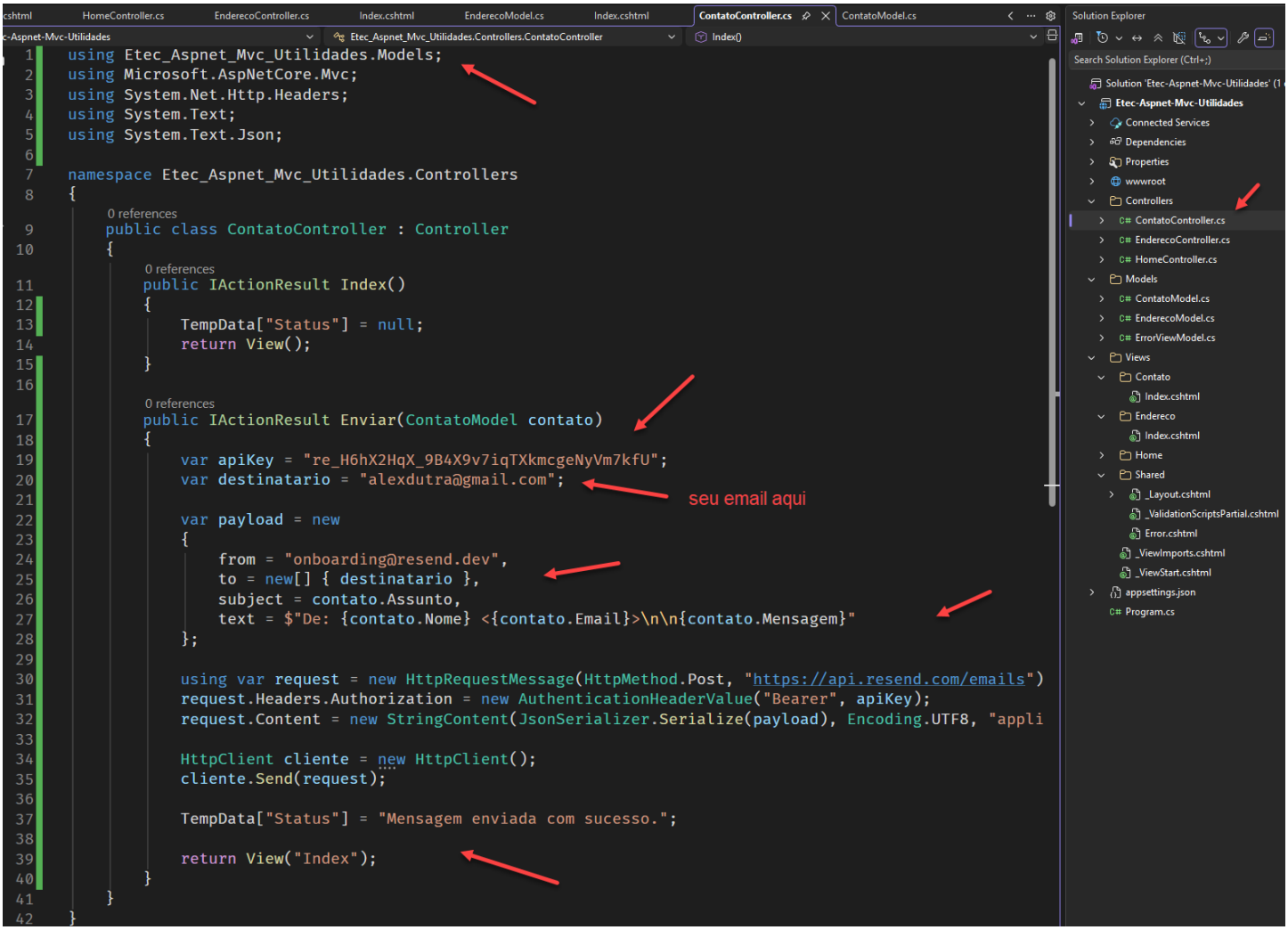
Agora vamos adicionar um novo **Controller**, onde iremos criar os códigos para envio dos dados do formulário para a API, clique o botão direito do mouse na pasta **Controllers**, para isso acesse as opções



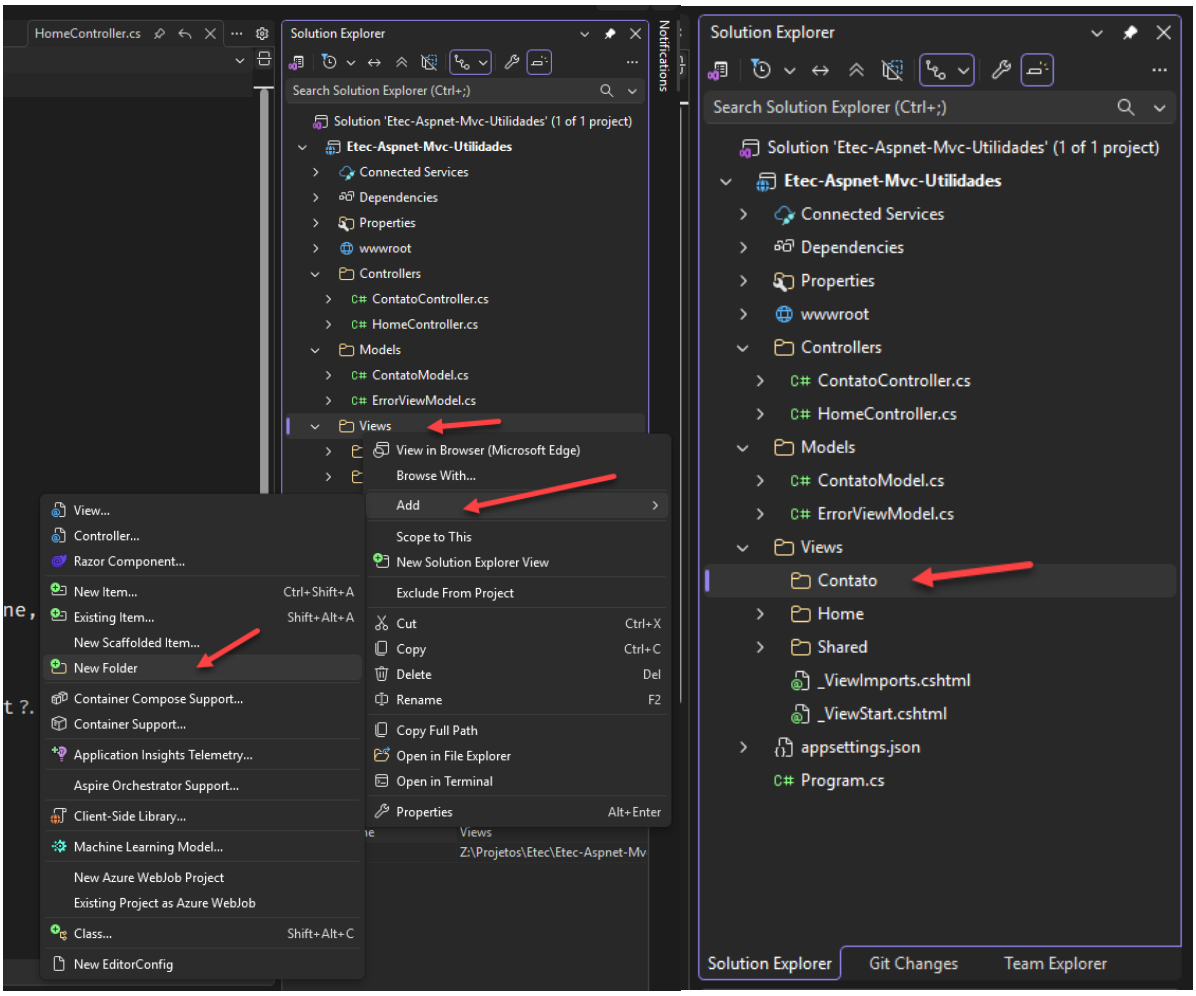




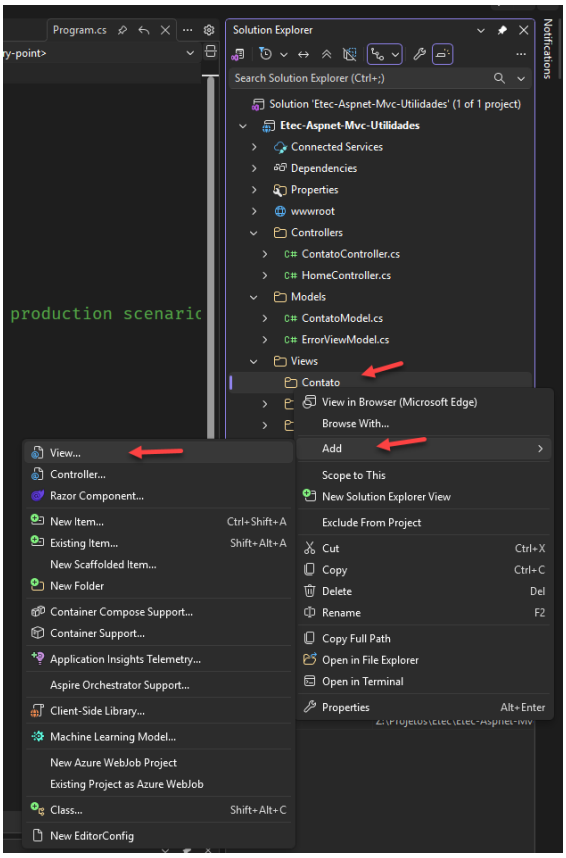
Vamos colocar o código para envio no **Controller** criado

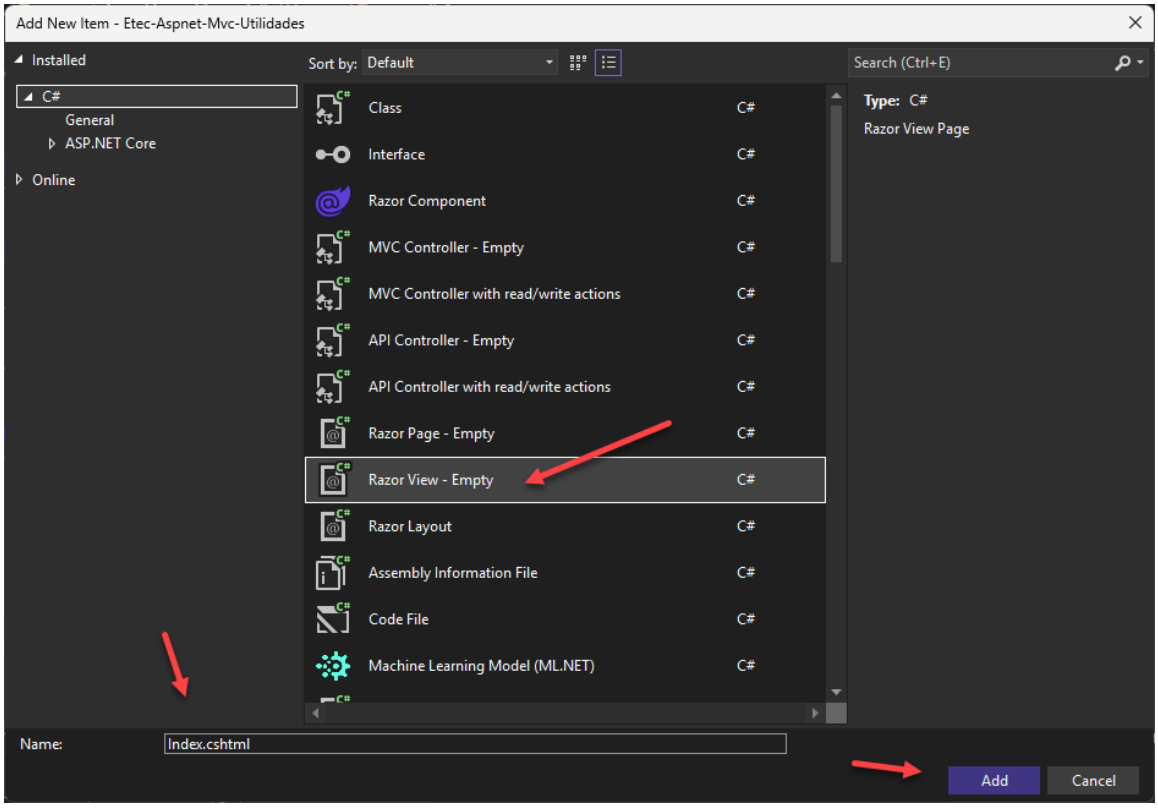


Agora vamos criar o formulário para digitação dos dados de contato, para isso acesse a pasta **Views** e clique com o botão direito, e crie uma nova pasta chamada **Contato**:



Agora na pasta Contato adicione uma nova **View** chamada **Index**, para isso clique o botão direito e acesse as opções:


Program.cs
Solution Explorer
Search Solution Explorer (Ctrl+;)
Solution 'Etec-Aspnet-Mvc-Utilidades' (1 of 1 project)
Etec-Aspnet-Mvc-Utilidades
Connected Services
Dependencies
Properties
wwwroot
Controllers
ContatoController.cs
HomeController.cs
Models
ContatoModel.cs
ErrorViewModel.cs
Views
Contato
production scenario
View in Browser (Microsoft Edge)
Browse With...
Add
Scope to This
New Solution Explorer View
Exclude From Project
Cut Ctrl+X
Copy Ctrl+C
Delete Del
Rename F2
Copy Full Path
Open in File Explorer
Open in Terminal
Properties Alt+Enter
View...
Controller...
Razor Component...
New Item... Ctrl+Shift+A
Existing Item... Shift+Alt+A
New Scaffolded Item...
New Folder
Container Compose Support...
Container Support...
Application Insights Telemetry...
Aspire Orchestrator Support...
Client-Side Library...
Machine Learning Model...
New Azure WebJob Project
Existing Project as Azure WebJob
Class... Shift+Alt+C
New EditorConfig


Add New Item - Etec-Aspnet-Mvc-Utilidades
Installed
C#
General
ASP.NET Core
Online
Sort by: Default
Search (Ctrl+E)
Class C#
Interface C#
Razor Component C#
MVC Controller - Empty C#
MVC Controller with read/write actions C#
API Controller - Empty C#
API Controller with read/write actions C#
Razor Page - Empty C#
Razor View - Empty C#
Razor Layout C#
Assembly Information File C#
Code File C#
Machine Learning Model (ML.NET) C#
Type: C#
Razor View Page
Name:
Index.cshtml
Add
Cancel


tml
Index.cshtml
ContatoController.cs
ContatoModel.cs
Etec-Aspnet-...des: Overview
@model Etec_Aspnet_Mvc_Utilidades.Models.ContatoModel
<h2>Contato</h2>
<form asp-controller="Contato" asp-action="Enviar" method="post">

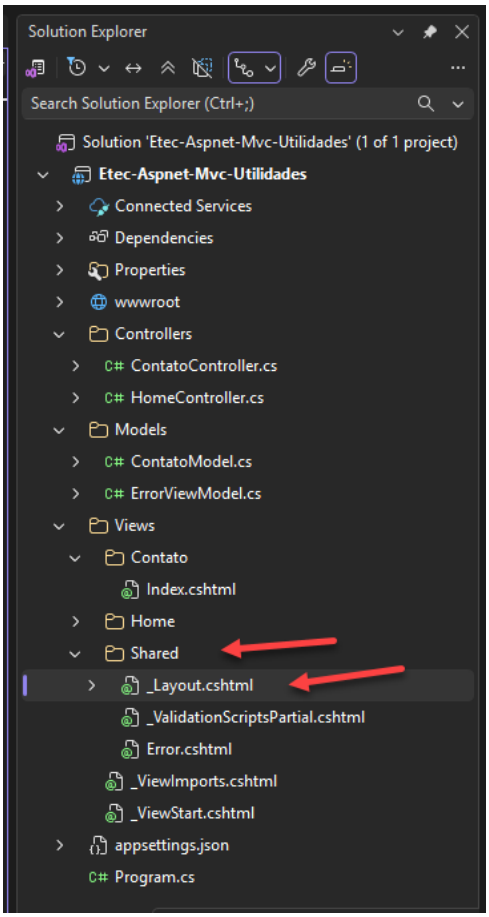
<label>Nome:</label>
<input asp-for="Nome" width="250" />

<label>E-mail:</label>
<input asp-for="Email" width="250" />

<label>Assunto:</label>
<input asp-for="Assunto" width="400" />

<label>Mensagem:</label>
<textarea asp-for="Mensagem" rows="4" cols="100"></textarea>

<button type="submit">Enviar</button>
@if (TempData["Status"] ≠ null)
{
<h3 class="text-success">@TempData["Status"]</h3>
}
</form>

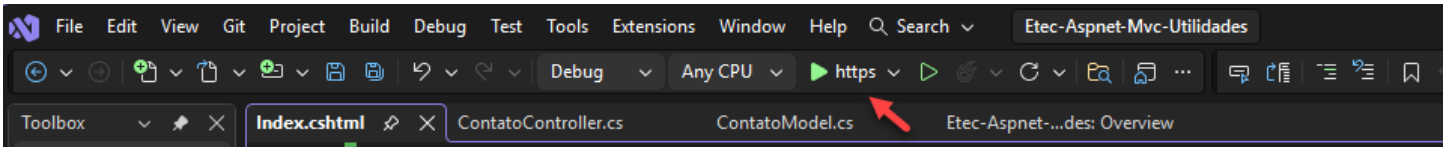


Agora adicione uma nova opção no Menu:

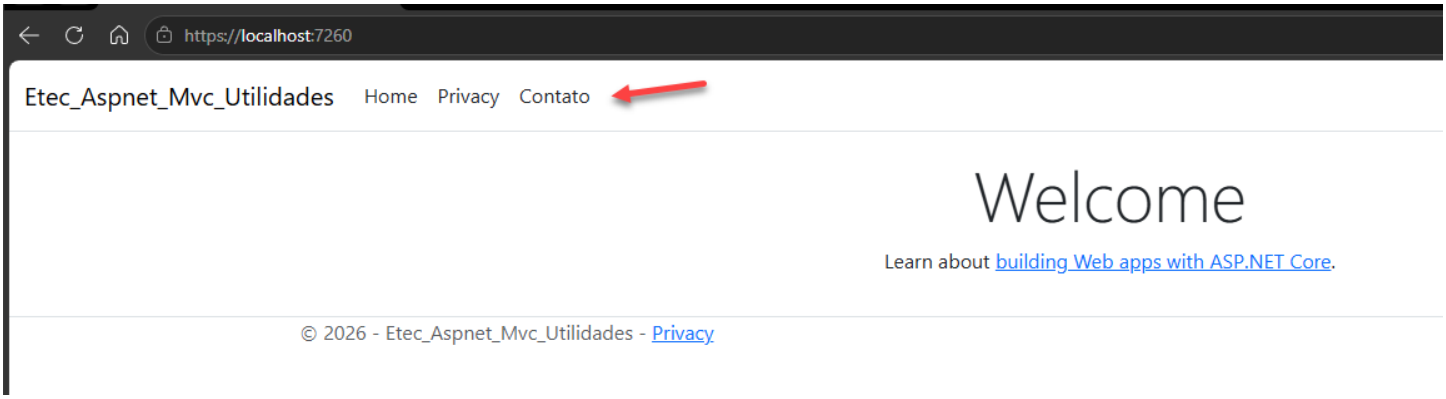
```
<header>
    <nav class="navbar navbar-expand-sm navbar-toggleable-sm navbar-light bg-white border-bottom box-shadow mb-3">
        <div class="container-fluid">
            <a class="navbar-brand" asp-area="" asp-controller="Home" asp-action="Index">Etec_Aspnet_Mvc_Utilidades</a>
            <button class="navbar-toggler" type="button" data-bs-toggle="collapse" data-bs-target=".navbar-collapse" aria-
                    aria-expanded="false" aria-label="Toggle navigation">
                <span class="navbar-toggler-icon"></span>
            </button>
            <div class="navbar-collapse collapse d-sm-inline-flex justify-content-between">
                <ul class="navbar-nav flex-grow-1">
                    <li class="nav-item">
                        <a class="nav-link text-dark" asp-area="" asp-controller="Home" asp-action="Index">Home</a>
                    </li>
                    <li class="nav-item">
                        <a class="nav-link text-dark" asp-area="" asp-controller="Home" asp-action="Privacy">Privacy</a>
                    </li>
                    <li class="nav-item">
                        <a class="nav-link text-dark" asp-area="" asp-controller="Contato" asp-action="Index">Contato</a>
                    </li>

                </ul>
            </div>
        </div>
    </nav>
</header>
```

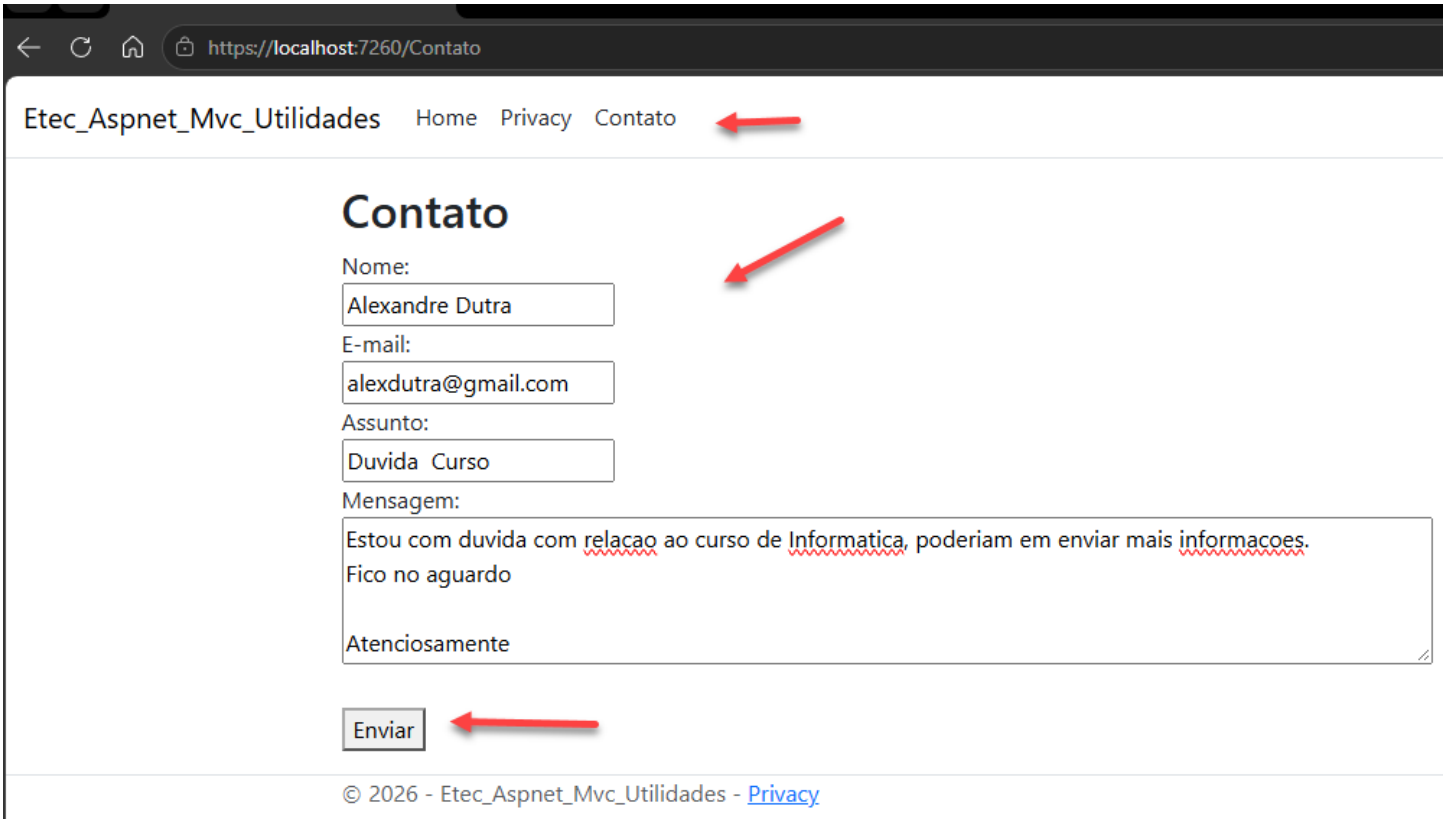
Agora clique na opção de executar o programa e aguarde a exibição do projeto:



Clique na opção **Contato**

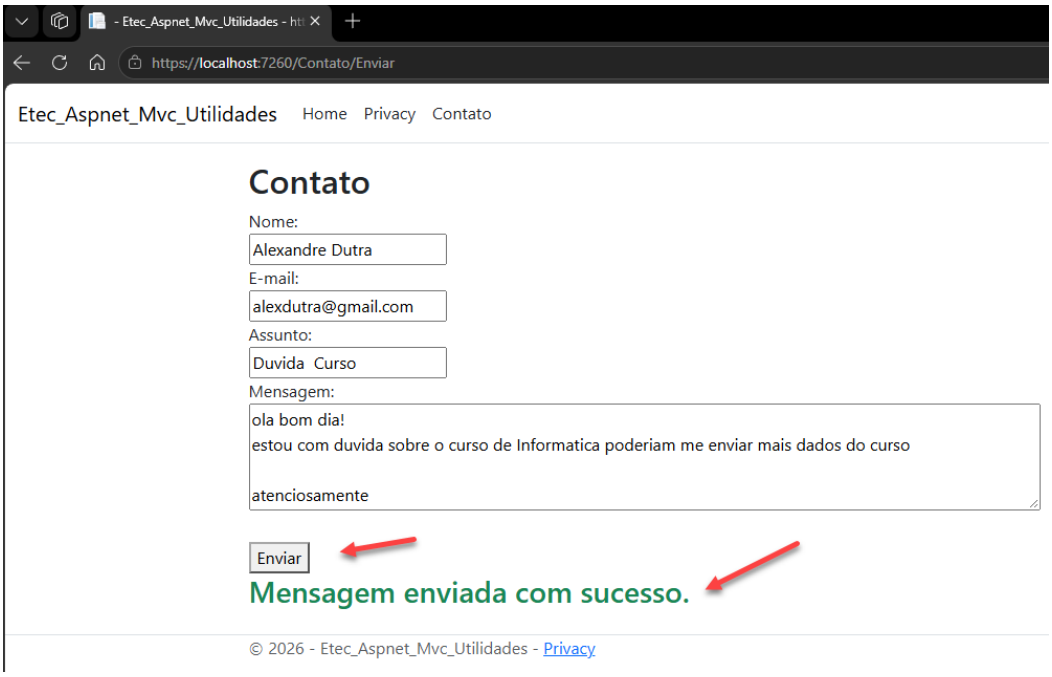


O formulário de **Contato** será exibido

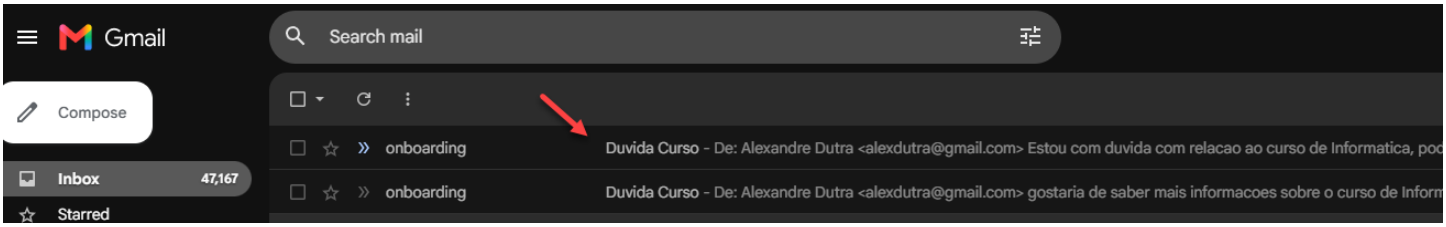


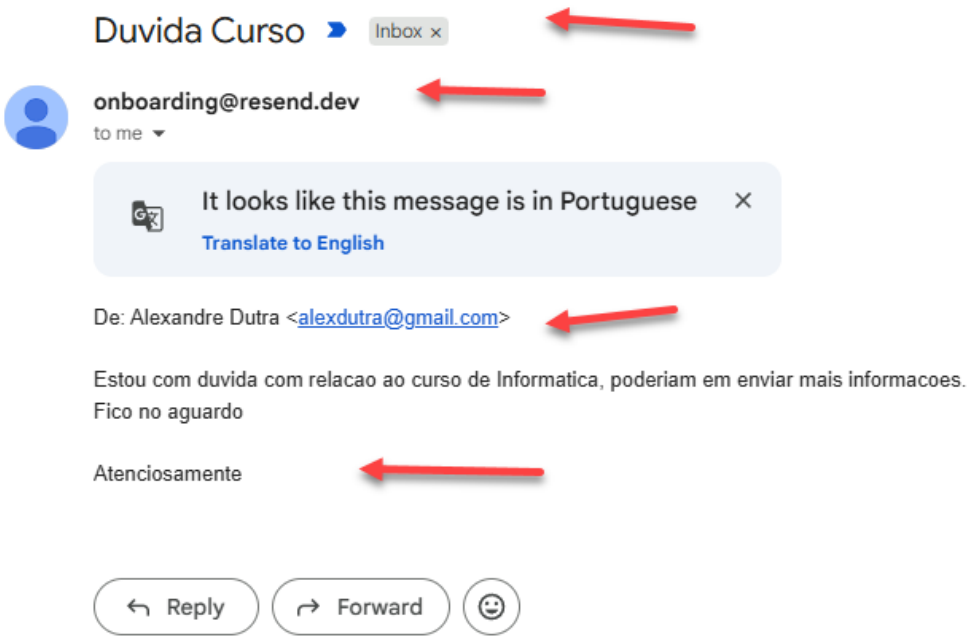
Digite as informações e clique em **ENVIAR.**

Após enviar será exibida a mensagem de sucesso

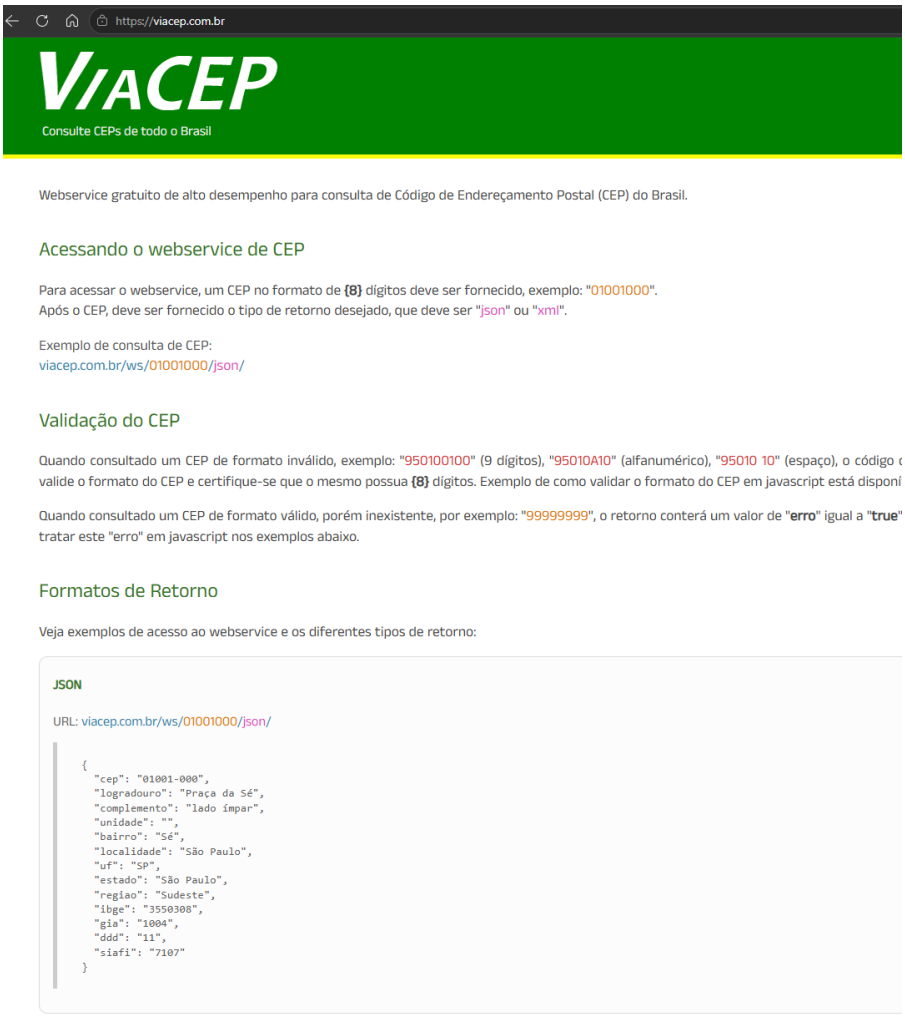


Agora abra seu email e verifique se o email chegou para você:

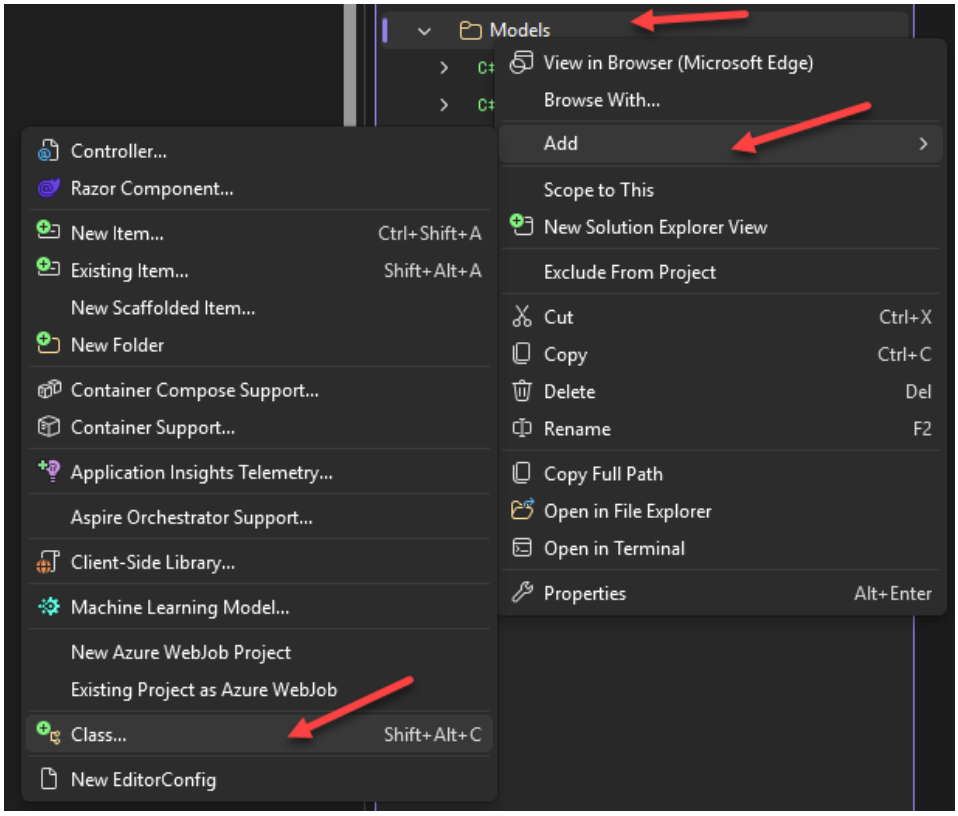


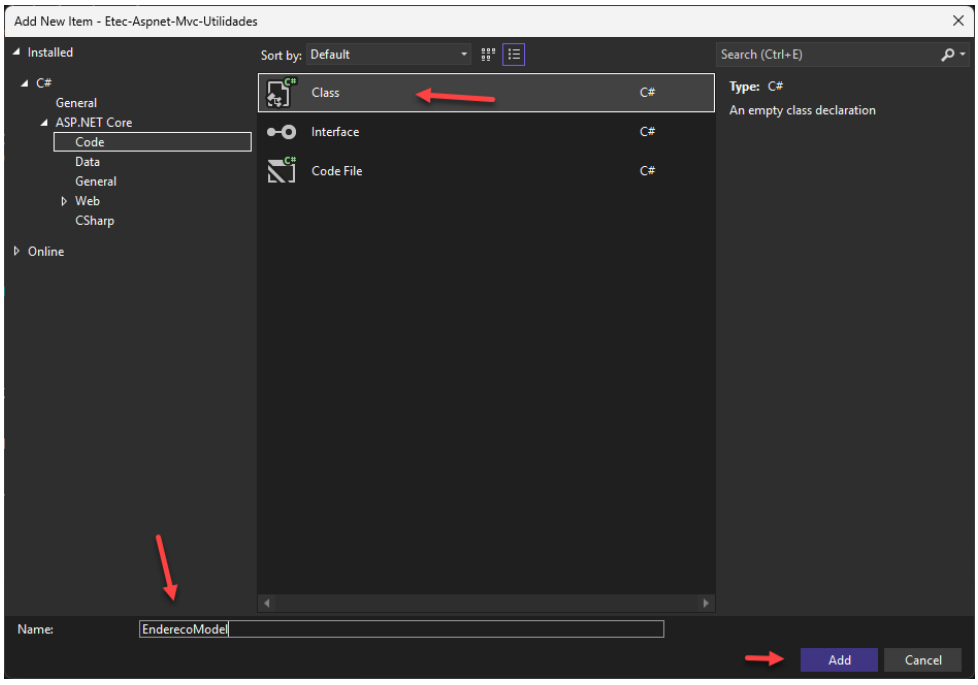


Agora vamos para criar uma nova funcionalidade para acessar uma api de consulta de CEPs, utilizaremos uma api gratuita chamada VIACEP: https://viacep.com.br/

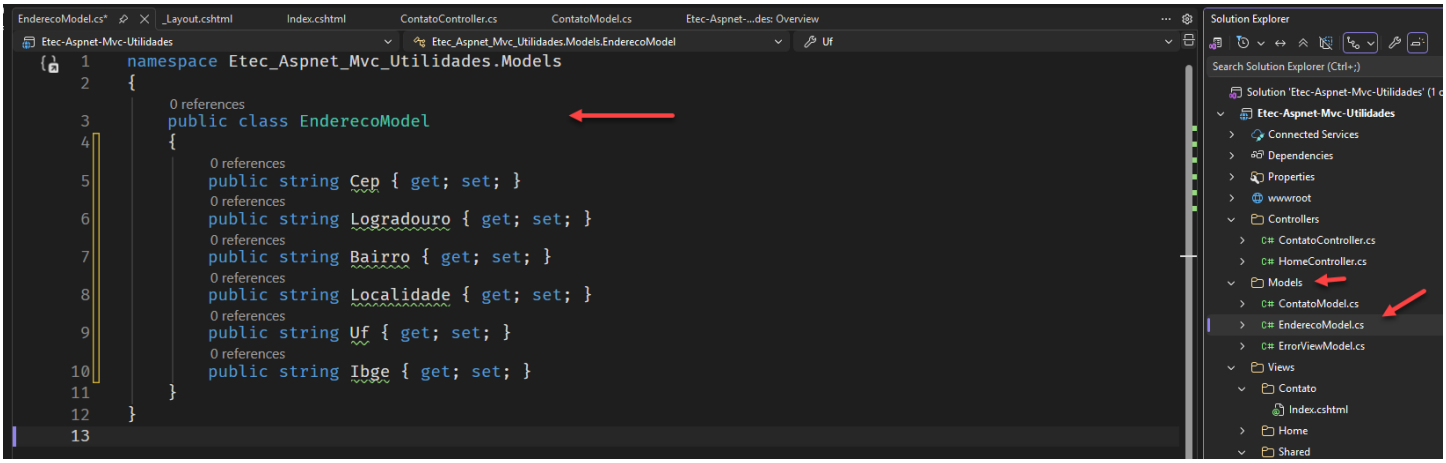


Adicione uma nova classe chamada **Models**:

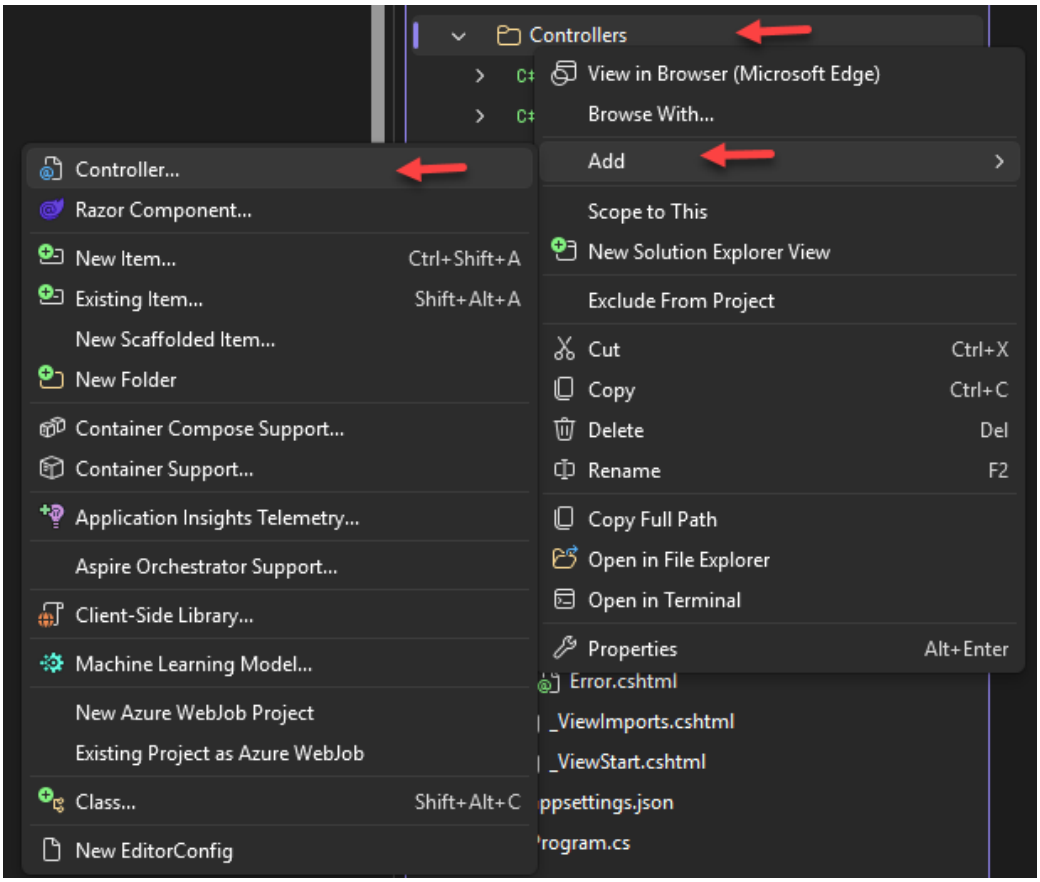


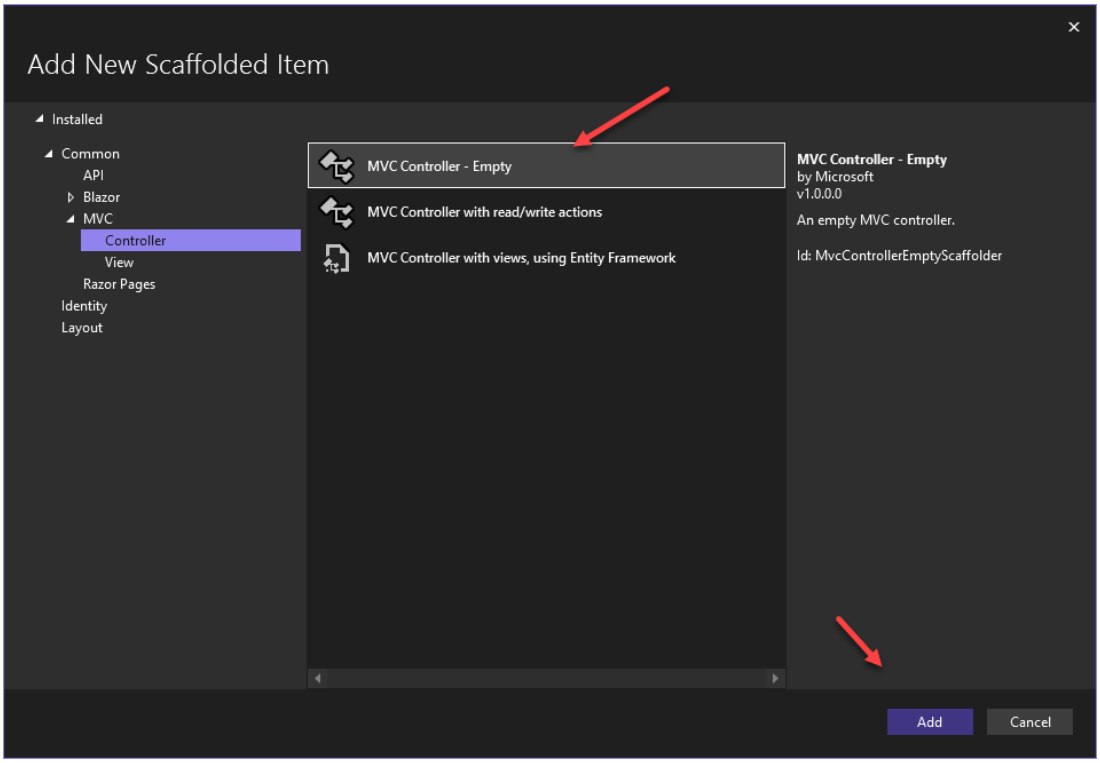


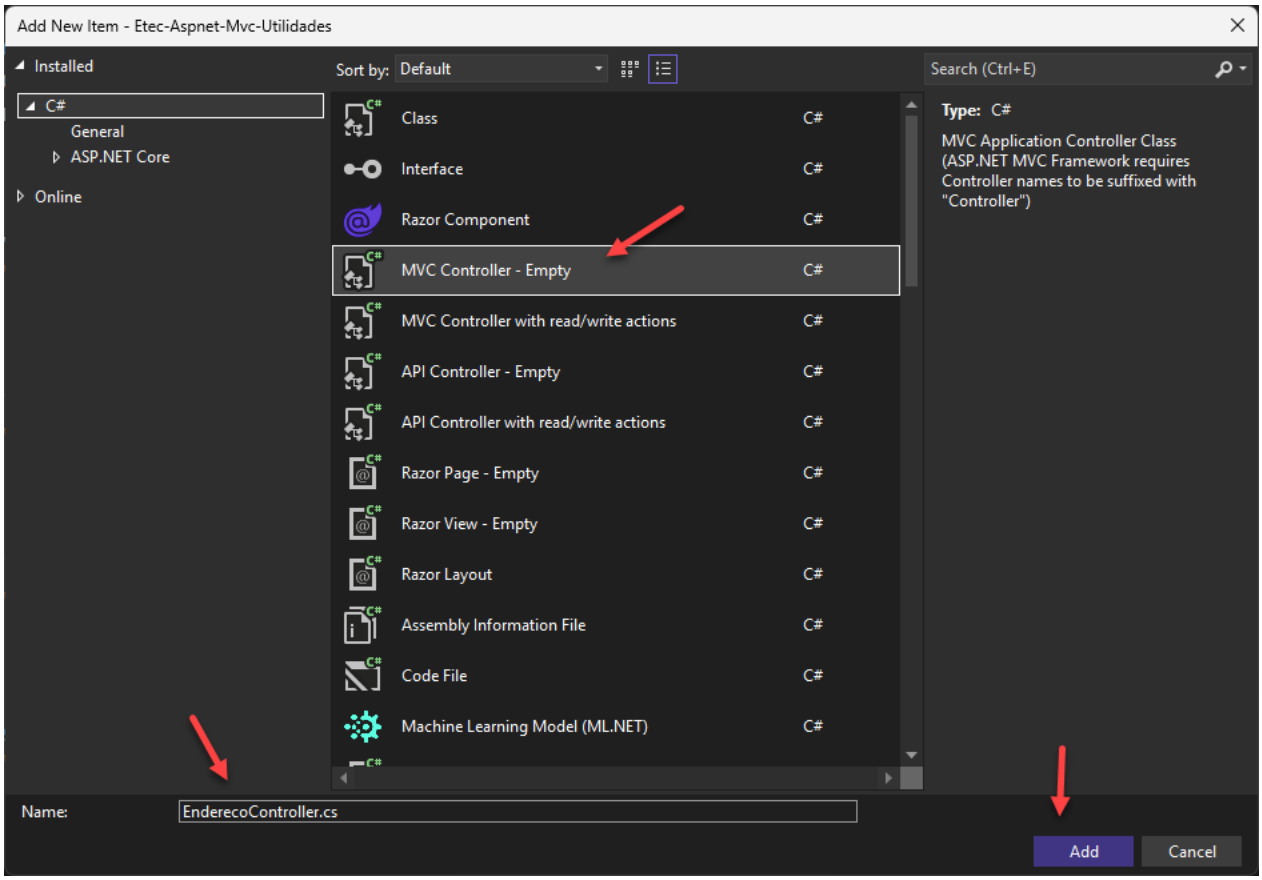
A classe deve ficar assim



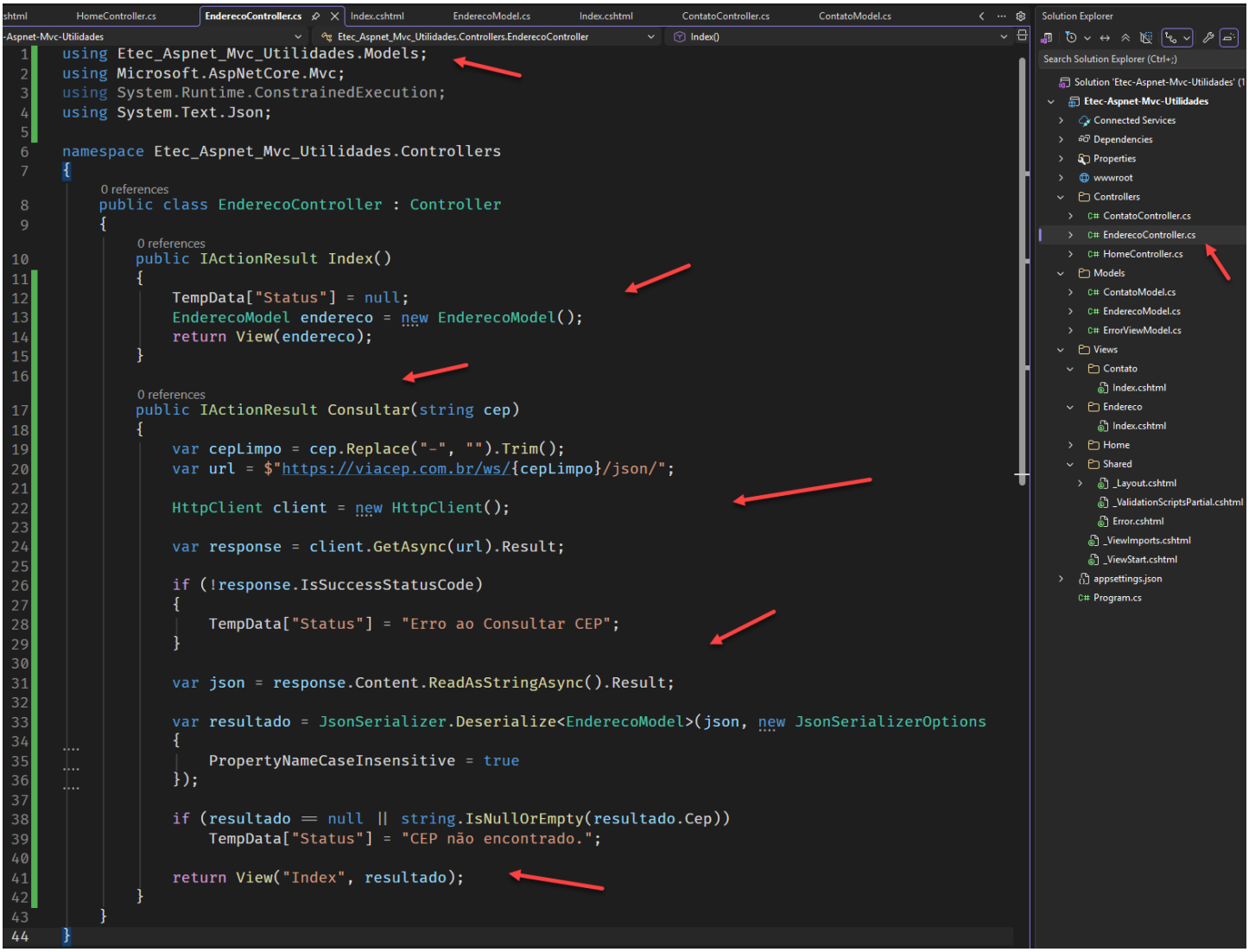
Agora vamos adicionar um novo **Controller** chamado **EnderecoController**:




Add New Scaffolded Item
Installed
Common
API
Blazor
MVC
Controller
View
Razor Pages
Identity
Layout
MVC Controller - Empty
MVC Controller with read/write actions
MVC Controller with views, using Entity Framework
MVC Controller - Empty
by Microsoft
v1.0.0.0
An empty MVC controller.
Id: MvcControllerEmptyScaffolder
Add
Cancel

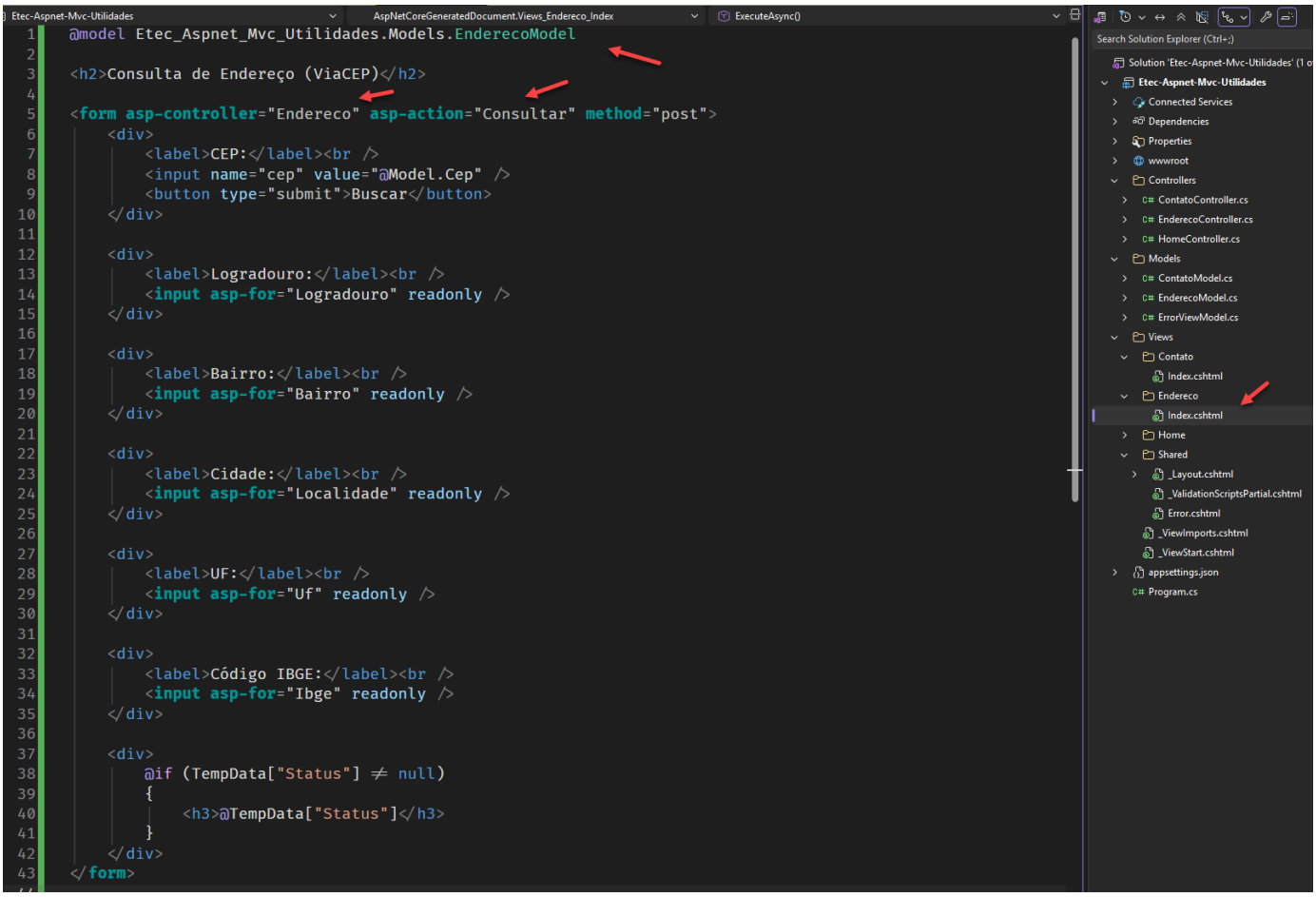

Add New Item - Etec-Aspnet-Mvc-Utilidades
Installed
C#
General
ASP.NET Core
Online
Sort by: Default
Search (Ctrl+E)
Class C#
Interface C#
Razor Component C#
MVC Controller - Empty C#
MVC Controller with read/write actions C#
API Controller - Empty C#
API Controller with read/write actions C#
Razor Page - Empty C#
Razor View - Empty C#
Razor Layout C#
Assembly Information File C#
Code File C#
Machine Learning Model (ML.NET) C#
Type: C#
MVC Application Controller Class (ASP.NET MVC Framework requires Controller names to be suffixed with "Controller")
Name: EnderecoController.cs
Add
Cancel

No **Controller** criado vamos adicionar uma nova ação de **Consultar**:



Agora vamos criar o formulário para informação e consulta do CEP:

Nas pasta **Views** adicione uma nova pasta chamada **Endereco** e dentro dela crie uma nova **view** chama **Index**:



Desafio:

Assim que concluir tente fazer agora um formulário parecido com o que fizemos com o CEP, porém agora ele deverá utilizar outra Api: https://cepify.com.br/ com retorno de CEP e outra Api chamada: https://goweather.xyz/v2/weather/ que retorna dados climáticos da cidade, segue link do projeto: https://github.com/robertoduessmann/weather-api. Para testar crie os links no menu superior conforme fizemos nos outros exemplos